



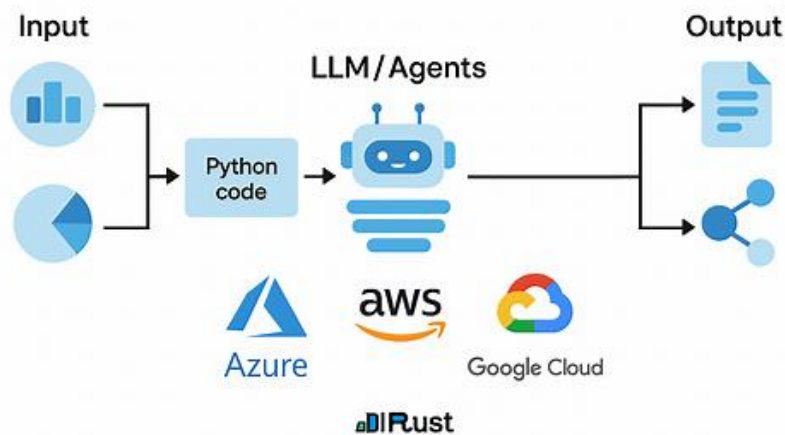


Voluntary Contribution

<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
 WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Predictive Analysis with Agents / LLMs



Customer Churn Propensity

REALTIME STREAM

AUTOMATIC RENEWAL

Customer Detail

Customer ID 771844

Age 40

Tenure 12 months

Monthly Spend \$76.29

Churn Score

92

Explanation

90% Model Accuracy

24% Selectivity

This customer, age 40, has been with us for one year and spends an average of \$76.29 per month. They had two support calls in the last year, indicating dissatisfaction. The customer shopped at a competitor recently, which increases churn risk. Hence, the churn score of the customer is high.

Suggestions

1. Reach out to address the issues raised in recent support calls.
2. Offer a personalized promotional discount to encourage loyalty.
3. Provide an exclusive benefit or service to differentiate from competitors.

Potential Churners	CHURN SCORE	VOUCHER	CALL FOLLOW-UP	IMPLEMENT BENEFITS
CUSTOMER ID	80	☐	☐	☐
658471	80	☐	☐	☐
567423	80	☐	☐	☐
879155	80	☐	☐	☐



Voluntary Contribution

<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python
Practice with MySQL on  AWS |  Azure |  GCP

project , for real: architecture, flow,
technology, SQL, Python,
LLMs/agents, and front-end.

1. Project Overview

Problem: Platform that predicts **customer churn** and the **probability of purchasing a new product** , using:

- Large transactional database in **MySQL** (~15M records)
- pipeline in **Python**
- **Agents/LLMs** for automation, explanations, and code generation.
- **API + Web front-end** for business teams

2. High-level architecture

Layer	Suggested technologies	Main function
Persistence	MySQL	Raw and aggregated data
Batch processing	Python, Pandas, Scikit-Learn, XGBoost	ETL, training, batch scoring
LLM/Agents	LLM API (OpenAI / Azure OpenAI) + LangChain	Intelligent orchestration, explanations
Service API	FastAPI / Flask	Present forecasts and reports
Front-end	React / Next.js / Vue	UI for business teams
Orchestration	Airflow / Prefect (optional)	Schedule jobs and workflows.
Monitoring	Logs + dashboards (Grafana/Metabase)	Metrics tracking



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

3. Data model and flow in MySQL

3.1 Main tables (example)

- **customers**
 - id_cliente , nome , cpf , data_cadastro , segmento , cidade , estado , renda , status
- **transactions** (may have tens of millions)
 - transaction_id , customer_id , transaction_date , value , category , channel
- **support_interactions**
 - id_interacao , id_cliente , data , tipo , descricao , status
- **churn_label**
 - client_id , churn (0/1), reference_data
- **score_churn** (model result)
 - id_cliente , score , data_score , modelo-versao
- **score_uplift_produtoX** (probability of buying product X)
 - id_cliente , score , data_score , modelo-versao

3.2 Data Flow

1. Operating systems (ERP, CRM, e-commerce) record data in MySQL transactional tables.
2. Scheduled Python jobs extract data in **batches** , filtered by date.
3. Scripts build **aggregated tables** (e.g., behavior over the last 3/6/12 months).
4. The ML pipeline consumes these aggregated tables and trains/updates models.
5. Scores (churn, purchase, etc.) are recorded back in MySQL.
6. The API reads these scores and serves them to the front-end.
7. LLM/Agent accesses MySQL indirectly via Python functions for ad hoc analysis.



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

4. Predictive pipeline in Python (offline/batch)

4.1 Extracting data from MySQL

```
python
import pandas as pd
import sqlalchemy as sa

engine = sa.create_engine(
    "mysql+pymysql://username:password@host:3306/database"
)

query = """
SELECT
c.id_cliente,
c.segment,
c.renda,
c.state,
ch.churn,
SUM(t.valor) AS valor_12m,
COUNT(t.id_transacao) AS qty_transacoes_12m,
MAX(t.data_transacao) AS ultima_compra,
COUNT(s.id_interacao) AS qty_complaints_12m
FROM clientes c
LEFT JOIN transacoes t
    ON c.id_cliente = t.id_cliente
    AND t.data_transacao >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
LEFT JOIN interacoes_suporte s
    ON c.id_cliente = s.id_cliente
    AND s.data >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
LEFT JOIN churn_label ch
    ON c.id_cliente = ch.id_cliente
WHERE c.data_cadastro <= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY c.id_cliente;
"""

df = pd.read_sql(query, engine)
```

4.2 Preparação e feature engineering

```
python
import numpy as np
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
df["average_value"] = df["value_12m"] /  
(df["number_of_transactions_12m"] + 1e-6)  
df["tem_reclamacao"] = (df["qtde_reclamacoes_12m"] > 0).astype(int)
```

```
Example: time since last purchase in days  
df["ultima_compra"] = pd.to_datetime(df["ultima_compra"])  
df["days_since_last_purchase"] = (  
    pd.Timestamp.today() - df["ultima_compra"]  
).dt.days.fillna(9999)
```

```
df_model = df.dropna(subset=["churn"])
```

4.3 Churn Model Training

python

```
from sklearn.model_selection import train_test_split  
from sklearn.metrics import roc_auc_score  
from xgboost import XGBClassifier  
  
features = [  
    "renda", "valor_12m", "qtde_transacoes_12m",  
    "valor_medio", "tem_reclamacao", "dias_desde_ultima_compra"  
]  
  
X = df_model[features]  
y = df_model["churn"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, stratify=y, random_state=42  
)  
  
model = XGBClassifier(  
    n_estimators=300,  
    max_depth=5,  
    learning_rate=0.05,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    eval_metric="logloss",  
)  
model.fit(X_train, y_train)  
  
probas_test = model.predict_proba(X_test)[: , 1]  
auc = roc_auc_score(y_test, probas_test)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MySQL on AWS | Azure | GCP

```
print("AUC:", auc)
```

4.4 Saving model and writing scores to MySQL

```
python
import joblib

joblib.dump(model, "model_churn_v1.pkl")

# Scoring of all customers (including without label yet)
df["score_churn"] = model.predict_proba(df[features])[:, 1]

# Recorded on scoreboard
df_scores = df[["customer_id", "score_churn"]].copy()
df_scores["data_score"] = pd.Timestamp.today().date()
df_scores["versao_model"] = "v1"

df_scores.to_sql(
    "score_churn",
    engine,
    if_exists="append",
    index=False
)
```

5. Agents/LLMs in practice

5.1 Functions (tools) that the agent can call

- `get_kpis_churn(data_referencia)` → MySQL query, returns metrics.
- `get_segmento_stats(segmento)` → statistics by segment.
- `get_clientes_risco_alto(top_n)` → lists clients with the highest score.
- `get_distribuicao_score()` → data for graphs.



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Example of a tool:

```
python
def get_segmento_stats(segmento: str):
    q = f"""
    SELECT
    c.segment,
        COUNT(*) AS qtd,
        AVG(s.score) AS score_medio,
        AVG(ch.churn) AS churn_real
    FROM clientes c
    JOIN score_churn s ON c.id_cliente = s.id_cliente
    LEFT JOIN churn_label ch ON c.id_cliente = ch.id_cliente
    WHERE c.segmento = '{segmento}'
    GROUP BY c.segment;
    """
    return pd.read_sql(q, engine).to_dict(orient="records")
```

5.2 Agente analítico com LangChain (exemplo conceitual)

```
python
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI

tools = [
    Tool(
        name="SegmentoStats",
        func=get_segment_stats,
        description="Returns churn statistics and score by segment"
    ),
    # other tools...
]

llm = OpenAI(temperature=0)

agent = initialize_agent(
    tools,
    llm,
    agent="zero-shot-react-description",
)

Question = "Explain churn behavior in the SME segment and suggest actions."
answer = agent.run(question)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
print(response)
```

The agent:

- interpret the question
- call `SegmentStats("PME")`
- receives raw data
- generates an analysis in natural language

5.3 Agents for AutoML/scheduled retraining

You can have an agent with the following role:

- “If AUC falls below 0.78 or drifts in > 2 features, retrain the model and generate a report explaining the changes.”

He would call, for example:

- `check_drift()`
- `retrain_model()`
- `compute_metrics()`
- And finally, I would use the LLM to write a report for the team.

6. Service API (FastAPI) for front-end and agents

6.1 Scoring/Query API

```
python
from fastAPI import FastAPI
import joblib
import pandas as pd
import sqlalchemy as sa

engine
sa.create_engine("mysql+pymysql://usuario:senha@host:3306/base")
modelo_churn = joblib.load("modelo_churn_v1.pkl")

app = FastAPI()
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
@app.get("/cliente/{id_cliente}")
def info_cliente(id_cliente: int):
    query = f"""
    SELECT c.*, s.score_churn
    FROM clientes c
    LEFT JOIN score_churn s ON c.id_cliente = s.id_cliente
    WHERE c.id_cliente = {id_cliente}
    ORDER BY s.data_score DESC
    LIMIT 1;
    """
    df = pd.read_sql(query, engine)
    if df.empty:
        return {"erro": "Cliente não encontrado"}
    return df.to_dict(orient="records")[0]
```

6.2 Endpoint that calls the LLM to explain the risk

python

```
from openai import OpenAI
```

```
client = OpenAI()
```

```
@app.get("/cliente/{id_cliente}/explicacao_churn")
def explicacao_churn(id_cliente: int):
    # Search for simplified feature and score data
    query = f"""
    SELECT
    c.id_cliente,
    c.segment,
    c.renda,
    c.state,
    s.score_churn,
    a.value_12m,
    a.qtde_transacoes_12m,
    a.qtde_reclamacoes_12m,
    a.days_since_last_purchase
    FROM clients c
    JOIN aggregates_client a ON c.id_client = a.id_client
    JOIN score_churn s ON c.client_id = s.client_id
    WHERE c.client_id = {client_id}
    ORDER BY s.data_score DESC
    LIMIT 1;
    """
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
df = pd.read_sql(query, engine)
se df.empty:
return {"error": "Client not found"}

data = df.to_dict(orient="records")[0]

prompt = f"""
You are a senior data analyst at a service company.
Explain, in clear and objective language, why this customer is at risk
of churn.

Customer data:
{data}
"""

resp = client.chat.completions.create(
model="gpt-4o-mini",
messages=[{"role": "user", "content": prompt}]
)

explanation = resp.choices[0].message["content"]
return {"client_id": customer_id, "explanation": explanation}
```

The front-end can display both the **score** and the **explanation in natural language** .

7. Front-end for business teams

7.1 Main Screens

1. **General dashboard**
 - Churn score distribution
 - Churn by segment / region
 - Top clients at greatest risk
2. **Customer screen**
 - Registration data
 - Churn score (with visual indicator)
 - Explanation generated via LLM
 - Suggested course of action (e.g., offer a benefit, contact customer service, etc.)



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

3. Campaign screen

- List of high-risk clients
- Filters by segment/region
- Export CSV for campaigns or CRM integration.

7.2 Simple example of an API call (React)

javascript

```
// Example in React (pseudo-code)
useEffect(() => {
  fetch(`/cliente/${idCliente}`)
    .then(res => res.json())
    .then(data => setCliente(data));

  fetch(`/cliente/${idCliente}/explicacao_churn`)
    .then(res => res.json())
    .then(data => setExplanation(data.explicacao));
}, [idClient]);
```

8. End-to-end flow (summarized)

1. **Data enters MySQL** (transactions, support, registration).
2. A scheduled **Python job generates aggregated tables and trains/updates models**.
3. Churn and purchase scores are recorded in `score tables*`.
4. **Agent/LLM** :
 - It helps to explore data, explain models, and generate reports.
 - It can trigger retraining in case of drift or performance drop.
5. **The API (FastAPI) exposes**:
 - customer information
 - scores
 - explanations in natural language
6. **The front-end consumes these APIs to**:
 - dashboards
 - customer perspective
 - support for campaigns and business decisions



Voluntary Contribution

<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python
Practice with MYSQL on  AWS |  Azure |  GCP

Real-time integration and deployment on Azure.

Check out my reasoning.

Let me set up a concrete scenario: **your real-time churn/propensity project running on Azure** , with:

- data ingestion
- model in production
- LLM/Agent supporting explanations and operations
- front-end calling everything via API

I will focus on: **architecture** , **recommended Azure services** , **integration** , and **deployment strategy** .

1. Real-time architecture on Azure (overview)

Layer	Suggested Azure service	Function
Database	Azure Database for MySQL	Store transactional data, aggregates, and scores.
Streaming/events	Azure Event Hubs or Service Bus	Receive events in near real-time (transactions, user actions)
Processing	Azure Functions / Azure Container Apps	Real-time scoring and business logic
ML Model	Azure Machine Learning / Container with API	Host churn/propensity model
LLM/Agents	Azure OpenAI + Functions/Counters	Explanations, automation, analysis, code/report generation
Backend API	Azure App Service / Containerized Applications	REST endpoints for front-end
Front-end	Static Web Apps / App Service (Node / React)	Web interface



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Orchestration	Azure Data Factory / Logic Apps / Function Timers	Batch jobs, retraining, pipelines
Monitoring	Application Insights + Log Analytics	Logs, metrics, alerts

2. Real-time data stream

2.1 Event Flow

1. The customer performs an action (purchase, login, complaint, etc.).
2. The application (or microservice) sends an **event** to **Azure Event Hubs** (or Service Bus).
3. An **Azure Function** is triggered when it receives the event:
 - consolidates features in real time (using MySQL + cache, if necessary);
 - **churn/propensity model** endpoint ;
 - saves the score (and context) in MySQL;
 - Optionally triggers notifications (email, Teams, etc.).

2.2 Front-end query flow

1. The user opens the client screen in the front-end (React, for example).
2. Front-end calls an endpoint in **Azure App Service / Container Apps** (`/api/cliente/{id}`).
3. API:
 - Reads data and scores from Azure MySQL;
 - Optionally calls **Azure OpenAI** to generate an **explanation** /recommendation in natural language;
 - Returns JSON with data + score + explanation.

3. Database: Azure Database for MySQL



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

You can migrate your on-premise MySQL or another provider to **Azure Database for MySQL – Flexible Server** .

- Tables similar to the ones we defined before:
 - customers , transactions , interacoes_suporte , churn_label , score_churn , score_uplift , etc.
- For real-time data, it's helpful to have a table of **ready-made features** and another of **recent scores** .

Example of a Python connection (in Azure Function or Container):

```
python
import sqlalchemy as sa

engine = sa.create_engine(
    "mysql+pymysql://username:password@my-
mysql.mysql.database.azure.com:3306/my_database"
)
```

4. Hosting the model on Azure

You have 2 main paths:

4.1 Azure Machine Learning (Managed Online Endpoint)

- You register your model (a .pkl or .onnx file).
- Creates a managed **online endpoint** with autoscaling.
- The function or API calls this endpoint via HTTP.

Example of a call (schematic):

```
python
import requests
import json

endpoint_url = "https://my-endpoint-
central.eastus.inference.ml.azure.com/score"
api_key = "YOUR_KEY"
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
def predict_churn(features_dict):
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {api_key}"
    }
    body = {"inputs": [features_dict]}
    resp = requests.post(endpoint_url, headers=headers,
        data=json.dumps(body))
    resp.raise_for_status()
    return resp.json()["predictions"][0]
```

4.2 Containerized model (Docker) running in:

- **Azure container applications** or
- **Azure Kubernetes Service (AKS)** (if more complex).

You expose a REST endpoint in the container and call it from the Functions/APIs.

5. Azure OpenAI + Agents

5.1 Classic use: explanations and reports

You create an **Azure OpenAI resource**, using a GPT-4o or similar template.

Example in Python (running in a Function or App Service):

```
python
from openai import AzureOpenAI

client = AzureOpenAI(
    api_key="YOUR_KEY",
    api_version="2024-02-15-preview",
    azure_endpoint="https://your-openai-resource.openai.azure.com"
)

def explain_churn(client_data: dict):
    prompt = f"""
    You are a data analyst. Explain, in business terms,
    Why is this client at risk of churn, and suggest practical actions to
    take.
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
Customer data:
{customer_data}
"""

resp = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}]
)
return resp.choices[0].message.content
```

5.2 Use as an agent (via LangChain / related libraries)

You can run the "agent" in a **Container App** or **Function** with:

- Tools for:
 - query MySQL,
 - call the model endpoint,
 - generate reports,
 - Sending emails (via Logic Apps or SendGrid).

This agent can be activated by:

- a **timer** (review performance daily);
- a **webhook** (called by the data team or external system);
- An integration with **Power Automate/Logic Apps** .

6. Azure Functions as real-time "glue"

6.1 Event-triggered function (Event Hubs)

Trigger: **EventHubTrigger**

Flow:

1. Receives event with `id_cliente` + context.
2. MySQL query to compose features (recent transactions, complaints, etc.).
3. It calls the churn model endpoint.
4. Save scores in MySQL.
5. Optional: send message to another topic (notification, queue, etc.).



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Esboço de função (Python):

```
python
import logging
import json
import sqlalchemy as sa
from azure.functions import EventHubEvent

engine = sa.create_engine("mysql+pymysql://...")

def main(event: EventHubEvent):
    body = event.get_body().decode("utf-8")
    dados = json.loads(body)
    id_cliente = dados["id_cliente"]

    # 1. montar features a partir do MySQL
    with engine.connect() as conn:
        df = pd.read_sql(f"""
            SELECT ...
            FROM ...
            WHERE id_cliente = {id_cliente}
        """, conn)
        features = df.iloc[0].to_dict()

    # 2. chamar modelo
    score = prever_churn(features)

    # 3. gravar score
    with engine.begin() as conn:
        conn.execute(
            sa.text("""
                INSERT INTO score_churn (id_cliente, score,
data_score, modelo_versao)
                VALUES (:id_cliente, :score, NOW(), :modelo_versao)
            """),
            {"id_cliente": id_cliente, "score": score,
"modelo_versao": "v1"}
        )

    logging.info(f"Score de churn para cliente {id_cliente}: {score}")
```

7. Backend (FastAPI/Flask) in Azure App Service or Container Apps



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

7.1 REST API for the front-end

You build a FastAPI API and deploy it to:

- **Azure App Service** (simpler), or
- **Container Apps** with Docker.

Example of an endpoint combining MySQL and Azure OpenAI:

```
python
from fastAPI import FastAPI
import sqlalchemy as sa
import pandas as pd

app = FastAPI()
engine = sa.create_engine("mysql+pymysql://...")

@app.get("/clientes/{id_cliente}")
def get_cliente(id_cliente: int):
    query = f"""
    SELECT c.*, s.score, s.data_score
    FROM clients c
    LEFT JOIN score_churn s ON c.client_id = s.client_id
    WHERE c.client_id = {client_id}
    ORDER BY s.data_score DESC
    LIMIT 1;
    """
    df = pd.read_sql(query, engine)
    if df.empty:
        return {"error": "Client not found"}
    return df.to_dict(orient="records")[0]

@app.get("/clientes/{id_cliente}/explicacao")
def explanation(client_id: int):
    # reuse get_client or make a new query with more features
    # and call Azure OpenAI
    ...
```

Deploy:



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Dockerfile → push to Azure Container Registry → create App Service / Container App pointing to the image.
- Or direct use of **App Service with Python** (without Docker).

8. Front-end on Azure

Options

- **Azure Static Web Apps** (ideal for static React/Next.js + separate API).
- **App Service** with Node/Next.js SSR.

Basic flow:

- The front end calls `/clientes/{id_cliente}` and `/clientes/{id_cliente}/explicacao`.
- Displays:
 - customer data,
 - churn score,
 - Text explanation generated via LLM,
 - eventual recommendations for action (also generated by the LLM).

9. Deployment and CI/CD

Azure Tools

- **GitHub Actions** or **Azure DevOps Pipelines**.
- Typical pipelines:
 1. **Data/Model Pipeline**
 - Training the model (can run on Azure ML or in a container).
 - Model registration.
 - Automated endpoint deployment (if it passes the tests).
 2. **Infrastructure/Backend/Front**
 - Build the Docker image (API and, if desired, the front end).
 - Push to Azure Container Registry.
 - Deploy in Container Apps / App Service.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

3. Infrastructure as Code

- Bicep / ARM / Terraform to create:
 - Azure Database for MySQL,
 - Event Hubs,
 - Azure Functions,
 - Container Apps / App Service
 - Azure OpenAI,
 - Application Insights.

10. Putting it all together: the “history” of the system

1. **Data is entered** into Azure MySQL (batch) and Event Hubs (real-time events).
2. **The churn model** is published on an endpoint (Azure ML or Container).
3. **Azure Functions** :
 - They react to events to perform scoring in (near) real-time.
 - They periodically send agents/LLMs to:
 - check performance,
 - generate reports,
 - suggest adjustments.
4. **Azure OpenAI** :
 - Generates explanations per client.
 - Summarizes key performance indicators for management.
 - Supports the data team (generating code, SQL, and analytics).
5. **API (FastAPI)** in App Service/Container Apps serves the front-end.
6. **The front-end** displays everything for the business: dashboards, customer details, recommendations.



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
WhatsApp 55 21 999618643

Voluntary Contribution

Predictive Analysis with Agents and LLMs in Action: Hands-On Python
Practice with MYSQL on **AWS** | **Azure** | **GCP**

Real-time integration and deployment on AWS

Check out my reasoning.

I'm going to build a scenario equivalent to Azure, but now **100% on AWS** , focused on **real-time deployment** , **MySQL integration** , **predictive modeling** , and **LLM/agents** .

1. Real-time architecture on AWS (overview)

Layer	Suggested AWS service	Function
Database	Amazon RDS for MySQL	Transactional data, aggregates, scores
Streaming/events	Amazon Kinesis or SQS data streams	Real-time event reception
Processing	AWS Lambda or AWS Fargate (ECS)	Real-time scoring, logic
ML Model	Amazon SageMaker Endpoint or	Endpoint of the churn/propensity





Voluntary Contribution

<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
 WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

	Fargate	model
LLM/Agents	Call to an LLM API (such as OpenAI)	Tutoring, analysis, automation
Backend API	Amazon API Gateway + Lambda or ECS	Expose REST endpoints
Front-end	S3 + CloudFront or Amplify	Web UI for business
Orchestration	Step Functions / EventBridge / Airflow	Scheduling, batch pipelines
Monitoring	CloudWatch + X-Ray + logs	Metrics and observability

2. General real-time flow

2.1 Business Events

- The application (e-commerce, CRM, internal app) publishes an event to **Kinesis** or sends a message to **SQS** :
 - Minimum data required: `client_id` , `event_type` , `timestamp` , `value` , etc.
- A **Lambda signal** is triggered (directly from Kinesis or the SQS queue).
- Lambda:
 - Queries the **RDS MySQL database** to retrieve customer history/aggregated data;
 - Assemble the feature vector;
 - It calls the **model endpoint** in SageMaker or the container;
 - `score_churn` table or similar);
 - Optionally, publish another event (for example, to notify the sales team).



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

2.2 Front-end query

1. The user accesses a page (React, Angular, etc.) hosted on **S3 + CloudFront** or **Amplify** .
2. The front calls an endpoint via **API Gateway** → Lambda/ECS.
3. The API:
 - Reads customer data and score in RDS;
 - It calls an external LLM (such as OpenAI) to generate an explanation in business language;
 - It returns JSON with all of that information to the front-end.

3. Database: Amazon RDS for MySQL

You configure a **MySQL RDS** with sufficient capacity to:

- Store 15M+ records (easy).
- Support read queries for real-time scoring.
- record scores.

Suggested tables (as before):

- customers
- transactions
- support_interactions
- churn_label
- score_churn
- intermediate aggregates (e.g., customer_aggregate_12m).

Typical connection in Python (within a Lambda or container):

```
python
import sqlalchemy as sa

engine = sa.create_engine(
    "mysql+pymysql://user:password@your-endpoint-rds:3306/your_base"
)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

4. Deploying the predictive model on AWS

4.1 Option A: Amazon SageMaker Endpoint

Flow:

1. You train locally (or in the SageMaker Notebook itself).
2. Saves the model (`joblib.dump(model, "model.pkl")`).
3. Go up to S3;
4. You create a **SageMaker endpoint** with this template;
5. The endpoint is available via HTTP within the VPC.

Endpoint call (simplified example):

```
python
import boto3
import json

sm = boto3.client("sagemaker-runtime", region_name="us-east-1")

def predict_churn(features_dict):
    response = sm.invoke_endpoint(
        EndpointName = "my-endpoint-churn",
        ContentType="application/json",
        Body=json.dumps({"features": features_dict})
    )
    result = json.loads(response["Body"].read().decode("utf-8"))
    return result["score"]
```

4.2 Option B: Container with model (ECS/Fargate)

You create an API (e.g., FastAPI) that loads the model and exposes ``/predict`` . You package it with Docker and run it on **ECS Fargate** behind an **Application Load Balancer** .

Lambda/Kinesis or the backend API simply makes a `requests.post` request to the internal service.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

5. Lambda for real-time scoring (integrating Kinesis + RDS + model)

Example of a Lambda function (trigger: Kinesis):

```
python
import json
import pandas as pd
import sqlalchemy as sa
import boto3

engine = sa.create_engine("mysql+pymysql://user:password@rds-
endpoint:3306/base")
sm = boto3.client("sagemaker-runtime", region_name="us-east-1")

def prever_churn(features_dict):
    response = sm.invoke_endpoint(
        EndpointName="meu-endpoint-churn",
        ContentType="application/json",
        Body=json.dumps({"features": features_dict})
    )
    result = json.loads(response["Body"].read().decode("utf-8"))
    return result["score"]

def lambda_handler(event, context):
    for record in event["Records"]:
        payload = json.loads(record["kinesis"]["data"])
        id_cliente = payload["id_cliente"]

        # 1. Buscar dados agregados do cliente no MySQL
        with engine.connect() as conn:
            df = pd.read_sql(f"""
                SELECT *
                FROM cliente_agregado_12m
                WHERE id_cliente = {id_cliente}
            """, conn)

        if df.empty:
            continue

        features = df.iloc[0].to_dict()

        # 2. Chama o modelo
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
score = prever_churn(features)

#3. Record score in the table.
with engine.begin() as conn:
    conn.execute(sa.text("""
INSERT INTO score_churn (client_id, score, data_score,
template_version)
VALUES (:id, :score, NOW(), :version)
"""), {"id": id_cliente, "score": score, "versao": "v1"})

return {"statusCode": 200}
```

6. LLM/Agents on AWS (using an external API such as OpenAI)

AWS doesn't yet have its own universal LLM like Azure OpenAI; so you typically use:

- Lambda/ECS OpenAI API.

6.1 Explanation function in natural language

```
python
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

def explain_churn(client_data: dict):
    prompt = f"""
You are a data analyst. Explain, in Portuguese and in business
language,
Why does this client present a risk of churn, and suggest 3 practical
actions.

Customer data:
{customer_data}
"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

You can call this in:

- an API endpoint (for the front end to request explanations on-demand),
- a scheduled Lambda (that generates reports)
- or a more complex “agent” running on ECS/Fargate.

6.2 Orchestrating agent (conceptual)

You can run an agent (with LangChain, for example) in a Fargate container:

- Agent tools:
 - `query_mysql(sql)`
 - `call_model(features)`
 - `write_report_to_s3(content)`

And then ask for things like:

"Generate a daily report with the 50 highest-risk clients, explaining the reasons by segment."

The agent:

1. Builds SQL, RDS query;
2. processes data;
3. Uses an LLM to write a report;
4. salva no S3 para times de negócio.

7. API backend: API Gateway + Lambda ou ECS

7.1 Exemplo: API com Lambda para servir front-end

Um endpoint típico:

- `GET /clientes/{id_cliente}`
- `GET /clientes/{id_cliente}/explicacao`



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Lambda (esqueleto):

```
python
import json
import pandas as pd
import sqlalchemy as sa
from openai import OpenAI

engine = sa.create_engine("mysql+pymysql://user:password@rds-
endpoint:3306/base")
client = OpenAI(api_key="SUA_API_KEY")

def lambda_handler(event, context):
    path = event["path"]
    http_method = event["httpMethod"]

    if http_method == "GET" and path.startswith("/clientes/") and
path.endswith("/explicacao"):
        # /clientes/{id}/explicacao
        id_cliente = int(path.split("/") [2])
        return explicacao_cliente(id_cliente)
    elif http_method == "GET" and path.startswith("/clientes/"):
        id_cliente = int(path.split("/") [2])
        return dados_cliente(id_cliente)
    else:
        return {"statusCode": 404, "body": "Not found"}

def dados_cliente(id_cliente: int):
    df = pd.read_sql(f"""
        SELECT c.*, s.score, s.data_score
        FROM clientes c
        LEFT JOIN score_churn s ON c.id_cliente = s.id_cliente
        WHERE c.id_cliente = {id_cliente}
        ORDER BY s.data_score DESC
        LIMIT 1;
        """, engine)

    if df.empty:
        return {"statusCode": 404, "body": "Cliente não encontrado"}

    return {
        "statusCode": 200,
        "body": json.dumps(df.to_dict(orient="records") [0],
default=str)
    }
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
def explicacao_cliente(id_cliente: int):
    # Busca dados mais ricos (features) para explicação
    df = pd.read_sql(f"""
SELECT a.*, s.score
FROM cliente_agregado_12m a
JOIN score_churn s ON a.client_id = s.client_id
WHERE a.client_id = {client_id}
ORDER BY s.data_score DESC
LIMIT 1;
""", engine)

    if df.empty:
        return {"statusCode": 404, "body": "Client not found"}

    data = df.to_dict(orient="records")[0]

    prompt = f"""
    Explain in business terms why this customer is at risk of churn.
    Using the fields below, suggest concrete actions:

    {data}
    """

    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )

    explicacao = resp.choices[0].message.content

    return {
        "statusCode": 200,
        "body": json.dumps({"id_cliente": id_cliente, "explicacao":
explicacao}, ensure_ascii=False)
    }
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

8. Front-end na AWS

Opção simples e robusta

- **React/Next.js** hospedado em **S3** (static website) + **CloudFront** como CDN.

O front:

- Reads `BASE_API_URL` from an environment variable;
- fetch data from the API Gateway API (e.g., `https://xyz.execute-api.us-east-1.amazonaws.com/prod/clientes/123`);
- displays:
 - customer data,
 - score,
 - explanation generated via LLM,
 - recommendations for action.

9. CI/CD on AWS

Tools

- **GitHub Actions** or **AWS CodePipeline** + **CodeBuild** .

Typical scenario:

1. **Infrastructure** + **code repository** (IaC with CloudFormation, CDK, or Terraform).
2. Pipeline:
 - Image build (if using containers).
 - Tests.
 - Deploy to ECS/Fargate / Lambda / API Gateway.
3. Another pipeline for:
 - Training the model (in SageMaker or a job container).
 - Model registration.
 - Endpoint update (blue/green deployment if necessary).



Predictive Analysis with Agents and LLMs in Action: Hands-On Python
Practice with MYSQL on  AWS |  Azure |  GCP

10. Tying everything together in the "history" of the system.

1. **Customer events** arrive via the application and are sent to **Kinesis/SQS** .
2. **lambda** scoring:
 - Build features using RDS.
 - Call the model in SageMaker,
 - Records score in RDS.
3. Business teams access a **front-end** in S3+CloudFront:
 - They see the client, the score, and the explanation generated via LLM.
4. An **agent/LLM** running on ECS/Lambda:
 - analyzes model performance metrics,
 - Detects a drop in performance or a change in behavior.
 - suggests retraining / new features / campaign adjustments.
 - It generates daily or weekly reports.

Real-time integration and deployment on the Google Cloud Platform.

Check out my reasoning.

I'll set you up a scenario equivalent to those of Azure and AWS, now **100% on Google Cloud Platform (GCP)** , focused on:

- (Almost) real time
- MySQL (Cloud SQL) with many records
- predictive model in production
- LLMs/Integrated Agents (Vertex AI)
- API + front-end consuming everything





PREMIUM

Pedro Paulo Ribeiro - PCD

Desenvolvedor desde 1987 e Cientista de Dados.

Voluntary Contribution

<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
 WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

I will organize it into: **architecture** , **data flow** , **GCP services** , and code examples.

1. Real-time architecture in GCP (overview)

Layer	Suggested GCP service	Function
Database	Cloud SQL for MySQL	Transactional data, aggregates, scores
Streaming/events	Pub/Sub	Real-time events (transactions, actions)
Processing	Cloud Functions / Cloud Run	Real-time scoring, logic
ML Model	Vertex AI (Endpoint) or Cloud Run	Endpoint of the churn/propensity model
LLM/Agents	Vertex AI Generative AI / Model Garden	Tutoring, automation, analytics
Backend API	Cloud Run / GKE + API Gateway / Load Balancer	REST endpoints for front-end
Front-end	Cloud Run (Next.js) or Firebase Hosting / Cloud Storage + CDN	UI for business teams
Orchestration	Cloud Composer (Airflow) / Workflows / Cloud Scheduler	Batch jobs, retraining, pipelines
Monitoring	Cloud Logging + Cloud Monitoring	Logs, metrics, alerts

2. Real-time data stream

2.1 Event Flow

- The application (e-commerce, CRM, internal app) publishes an event in **Pub/Sub** with:
 - `id_cliente` , `tipo_evento` , `timestamp` , `valor` , etc.
- A **Cloud Function** (trigger Pub/Sub) is activated for each event.
- A Function:
 - Queries **Cloud SQL for MySQL** to assemble the feature vector;
 - Calls the model endpoint in **Vertex AI Endpoint** (or API in Cloud Run);
 - writes the score to the Cloud SQL `score_churn` table;
 - Optionally, publish another event (e.g., for marketing automation).



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

2.2 Front-end query flow

1. The user opens the client screen on the front-end (React/Next.js).
2. The front-end calls an **API in Cloud Run** (or GKE) via HTTPS.
3. The API:
 - Reads data and scores from Cloud SQL;
 - Optionally calls an LLM in **Vertex AI Generative AI** to generate a user-friendly explanation of the risk;
 - Returns consolidated JSON.

3. Database: Cloud SQL for MySQL

You set up a **Cloud SQL for MySQL** (replicated/appropriate region).

Tables (as in the previous example):

- customers
- transactions
- support_interactions
- churn_label
- score_churn
- aggregated_client_12m (views or materialized tables with aggregated features)

Typical Python connection (in Cloud Run or Cloud Function with VPC Connector):

```
python
import sqlalchemy as sa

# Example with TCP connection (within the same VPC)
engine = sa.create_engine(
    "mysql+pymysql://username:password@CLOUD_IP_ADDRESS:3306/my_database"
)
```

Cloud SQL Auth Proxy or **Unix socket connection** is commonly used ; but the logic is the same.)



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

4. Predictive model in GCP

4.1 Option A: Vertex AI – Model Endpoint

Flow:

1. You can train the model locally or in Vertex AI (Notebook, pipeline).
2. Saved in a compatible format (e.g., `model.pkl` + wrapper or exported model).
3. Deploy to **Vertex AI Endpoint**.
4. The Cloud Function / Cloud Run calls the endpoint via SDK/REST.

Simplified example of a Python call:

```
python
from google.cloud import aiplatform

aiplatform.init(project="YOUR_PROJECT", location="us-central1")

endpoint = aiplatform.Endpoint("projects/SEU_PROJETO/locations/us-
central1/endpoints/ID_DO_ENDPOINT")

def predict_churn(features_dict):
    instances = [features_dict]
    prediction = endpoint.predict(instances=instances)
    Assuming it returns a list of scores
    score = prediction.predictions[0]
    return score
```

4.2 Option B: Cloud Run Model API

You package your model (FastAPI + `model.pkl`) into a Docker container and upload it to:

- **Cloud Run** (serverless, auto-scalable).

Cloud Run exposes `POST /predict`, and the internal functions/API call it via HTTPS.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

5. Cloud Function for real-time scoring (Pub/Sub → Cloud SQL → Vertex AI → Cloud SQL)

Conceptual example (Python):

```
python
import base64
import json
import pandas as pd
import sqlalchemy as sa
from google.cloud import aiplatform

# Inicialização global (fora da função)
engine = sa.create_engine("mysql+pymysql://usuario:senha@IP_DO_CLOUDSQL:3306/base")

aiplatform.init(project="SEU_PROJETO", location="us-central1")
endpoint = aiplatform.Endpoint("projects/SEU_PROJETO/locations/us-central1/endpoints/ID_DO_ENDPOINT")

def prever_churn(features_dict):
    instances = [features_dict]
    prediction = endpoint.predict(instances=instances)
    return prediction.predictions[0]

def pubsub_to_scoring(event, context):
    # Mensagem vem em base64
    if "data" not in event:
        return
    payload = json.loads(base64.b64decode(event["data"]).decode("utf-8"))

    id_cliente = payload["id_cliente"]

    # 1. Buscar dados agregados do cliente
    with engine.connect() as conn:
        df = pd.read_sql(f"""
            SELECT *
            FROM cliente_agregado_12m
            WHERE id_cliente = {id_cliente}
            """, conn)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
if df.empty:
    return

features = df.iloc[0].to_dict()

# 2. Chamar modelo no Vertex
score = prever_churn(features)

#3. Record scoring in Cloud SQL
with engine.begin() as conn:
    conn.execute(
        sa.text("""
INSERT INTO score_churn (client_id, score, data_score,
template_version)
VALUES (:id, :score, NOW(), :version)
"""),
        {"id": client_id, "score": score, "version": "v1"}
    )
```

This Cloud Function is associated with a Pub/Sub topic. Whenever a new client event is published, the flow rotates.

6. LLM/Agents with Vertex AI

6.1 Explanation of churn with Vertex AI Generative AI

You can use **Vertex AI – Model Garden / Gemini** to generate explanations in natural language.

Conceptual example using generative:

```
python
from google.cloud import aiplatform

aiplatform.init(project="YOUR_PROJECT", location="us-central1")

def explain_churn(customer_data: dict):
    from vertexai.generative_models import GenerativeModel

    model = GenerativeModel("gemini-1.5-pro") # model name may vary
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
prompt = f"""
You are a data analyst at a service company.
Explain, in clear Portuguese, why this customer is at risk of churn.
and suggest 3 practical actions to reduce this risk.
```

```
Customer data:
{customer_data}
"""
```

```
response = model.generate_content(prompt)
return response.text
```

You can call this function:

- within an API (endpoint `/clientes/{id}/explicacao`), or
- periodically, to generate batch reports.

6.2 Agents (orchestration with LLM + tools)

You can build **agents** in Cloud Run/GKE using libraries like LangChain, with tools such as:

- `query_mysql(sql)` → reads Cloud SQL
- `call_model(features)` → calls Vertex AI
- `write_report_to_gcs(content)` → saves report to Cloud Storage

The idea: a container running a "smart orchestrator" that responds to commands or executes routines (via Cloud Scheduler + HTTP).



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

7. Backend (API) in Cloud Run

7.1 REST API for front-end

An API using FastAPI, for example, running on Cloud Run:

```
python
from fastAPI import FastAPI
import sqlalchemy as sa
import pandas as pd

app = FastAPI()
engine = sa.create_engine("mysql+pymysql://usuario:senha@IP_DO_CLOUDSQL:3306/base")

@app.get("/clientes/{id_cliente}")
def get_cliente(id_cliente: int):
    df = pd.read_sql(f"""
    SELECT c.*, s.score, s.data_score
    FROM clients c
           LEFT JOIN score_churn s ON c.id_cliente = s.id_cliente
    WHERE c.id_cliente = {id_cliente}
    ORDER BY s.data_score DESC
    LIMIT 1;
    """, engine)

    if df.empty:
        return {"erro": "Cliente não encontrado"}

    return df.to_dict(orient="records")[0]
```

7.2 Endpoint com explicação do LLM

```
python
from google.cloud import aiplatform
from vertexai.generative_models import GenerativeModel

aiplatform.init(project="SEU_PROJETO", location="us-central1")
model = GenerativeModel("gemini-1.5-pro")

@app.get("/clientes/{id_cliente}/explicacao")
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
def explicacao_cliente(id_cliente: int):
    df = pd.read_sql(f"""
    SELECT a.*, s.score
    FROM cliente_agregado_12m a
    JOIN score_churn s ON a.client_id = s.client_id
    WHERE a.client_id = {client_id}
    ORDER BY s.data_score DESC
    LIMIT 1;
    """, engine)

    if df.empty:
        return {"error": "Client not found"}

    data = df.to_dict(orient="records")[0]

    prompt = f"""
    Explain in business terms why this customer is at risk of churn.
    Using the fields below, suggest concrete actions:

    {data}
    """

    response = model.generate_content(prompt)
    return {"client_id": customer_id, "explanation": response.text}
```

Cloud Run is exposed via HTTPS; you can use:

- directly to the service URL, or
- Place an **API Gateway / HTTP Load Balancer** in front.

8. Front-end on GCP

Options

- **Cloud Run** with Next.js/React (SSR or SPA).
- **Firebase Hosting** (great for SPAs) + API calls in Cloud Run.
- **Cloud Storage** + **CDN** (Static site hosting).



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

The front line is:

- Call `/clients/{id}` for data + score;
- Call `/clients/{id}/explanation` for explanatory text;
- Display this on a risk dashboard, customer list, etc.

9. Orchestration and retraining

9.1 Training and retraining of the model

GCP Tools:

- **Vertex AI Pipelines** (based on KFP)
- **Cloud Composer (Managed Airflow)**
- **Cloud Workflows + Cloud Run/Functions**
- **Cloud Scheduler + Cloud Run/Function** for periodic jobs

You can:

1. Having a daily/weekly job that:
 - Extracts new data windows from Cloud SQL;
 - train a new model;
 - evaluates metrics;
 - If it's better than the current one, it automatically deploys to the Vertex AI Endpoint.
2. An agent (LLM) helps generate reports on:
 - decline or improvement in performance,
 - changes in customer behavior,
 - Suggestions for new features.

10. End-to-end history in GCP

1. **Event data** (transactions, customer actions) arrive in **Pub/Sub**.
2. **Cloud Function** consumes events:
 - Build features with Cloud SQL.
 - Call model in Vertex AI,



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Records scores in Cloud SQL.
- 3. **Front-end** (Cloud Run/Firebase Hosting):
 - query an API in Cloud Run,
 - Displays customer data, churn score, and explanation generated by LLM.
- 4. An **agent/LLM** running on Cloud Run:
 - Generates daily/weekly reports.
 - analyzes performance, drift, score distribution,
 - It produces recommendations for the business team.
- 5. **Pipelines in Vertex AI/Composer** :
 - retrain models,
 - versionam,
 - They are deploying new endpoints.

[Here is a complete guide to Predictive Analytics.](#)

PREDICTIVE ANALYSIS WITH AGENTS AND LLMs

Language Models, Intelligent Automation, and Best Practices with Python

Preface

Predictive analytics has evolved from traditional statistical models to intelligent systems capable of learning, adapting, and interacting with humans. Today, autonomous agents and language models (LLMs) expand the reach of data science, enabling automated pipelines, code generation, explainability, and integration with complex systems.

This book presents a complete journey: from the fundamentals of predictive analytics to building intelligent agents that use LLMs to predict, decide, and act. All with practical examples in Python.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

CHAPTER 1 — Fundamentals of Predictive Analytics

1.1 What is Predictive Analytics?

Predictive analytics uses historical data to predict future events. It combines:

- Statistic
- Machine Learning
- Data engineering
- Probabilistic models
- LLMs as cognitive agents

1.2 Types of Predictive Models

- **Regression** (predicting continuous values)
- **Classification** (categories)
- **Time series**
- **Generative Models** (LLMs)
- **Hybrid models** (LLM + traditional ML)

1.3 The Role of LLMs in Predictive Analytics

LLMs can:

- Create ML pipelines automatically.
- Explain models
- Generate features
- Create autonomous agents that train, evaluate, and implement models.
- Integrate unstructured data (text, logs, PDFs)



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

CHAPTER 2 — Data Preparation

2.1 Collection and Cleaning

Simple example in Python:

```
python
import pandas as pd

df = pd.read_csv("dados.csv")
df = df.dropna()
df = df[df["idade"] > 0]
```

2.2 Feature Engineering

```
python
df["renda_per_capita"] = df["renda"] / df["num_moradores"]
```

2.3 Normalization

```
python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
```

CHAPTER 3 — Traditional Predictive Models

3.1 Linear Regression

```
python
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
from sklearn.linear_model import LinearRegression

X = df[["age", "income"]]
y = df["gasto"]

model = LinearRegression()
model.fit(X, y)
```

3.2 Decision Trees

```
python
from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier()
clf.fit(X, y)
```

3.3 Random Forest

```
python
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=200)
rf.fit(X, y)
```

CHAPTER 4 — Introduction to LLMs

4.1 What are Language Models?

LLMs are neural networks trained on large volumes of text to predict the next word. But today they do much more:

- Reasoning
- Planning
- Task execution
- Code generation



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Data analysis

4.2 How LLMs help in Predictive Analytics

- They automate experiments.
- They create models on demand.
- They interpret the results.
- They generate insights.

CHAPTER 5 — Intelligent Agents

5.1 What is an Agent?

An agent is a system that:

- Observe
- Decide
- Act
- Learn from the environment.

5.2 Agents + LLMs

LLMs enable agents to:

- Self-employed
- Capable of planning
- With memory
- Using external tools (Python, APIs, databases)

5.3 Agent Frameworks

- LangChain
- AutoGen
- CrewAI



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- LlamaIndex

CHAPTER 6 — Building a Predictive Agent with Python

6.1 Basic Setup

```
python
!pip install langchain openai
```

6.2 Creating an agent that generates models

```
python
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI

def train_model(data):
    import pandas as pd
    from sklearn.linear_model import LinearRegression

    df = pd.read_csv(data)
    X = df.drop("target", axis=1)
    y = df["target"]

    model = LinearRegression()
    model.fit(X, y)
    Return "Model trained successfully!"

tools = [
    Tool(
        name="TrainModel",
        func=train_model,
        description="Trains a predictive model"
    )
]
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
]

llm = OpenAI()
agent = initialize_agent(tools, llm, agent="zero-shot-react-
description")

agent.run("Train a model using the data.csv file")
```

CHAPTER 7 — LLMs for Feature Engineering

7.1 Generating Features with LLM

```
python
prompt = """
Create 10 new features for a bank churn dataset.
"""

print(llm(prompt))
```

7.2 Transforming text into features

```
python
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X_text = vectorizer.fit_transform(df["commentarios"])
```

CHAPTER 8 — Time Series with Agents

8.1 Prediction with Prophet

```
python
from prophet import Prophet

df_prophet = df.rename(columns={"data": "ds", "sales": "y"})
model = Prophet()
model.fit(df_prophet)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

8.2 Agent that adjusts hyperparameters

The agent can:

- Test for seasonality
- Adjust periods
- Compare metrics

CHAPTER 9 — Model Evaluation

9.1 Metrics

- RMSE
- MOTHER
- AUC
- F1-score

9.2 Code

```
python
from sklearn.metrics import mean_squared_error

pred = model.predict(X)
rmse = mean_squared_error(y, pred, squared=False)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

CHAPTER 10 — Deployment with Agents

10.1 Agent that monitors drift

- Detects changes in the data.
- Retraining models
- Generates alerts

10.2 Example

```
python
def detect_drift(df_new, df_training):
    from scipy.stats import ks_2samp
    results = {}
    for col in df_treino.columns:
        results[col] = ks_2samp(df_train[col], df_new[col]).pvalue
    return results
```

CHAPTER 11 — Real Cases

- Demand forecast
- Fraud detection
- Churn
- Credit risk
- Predictive maintenance
- Text analysis with LLMs



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

CHAPTER 12 — The Future of Predictive Analytics

- Fully autonomous agents
- Domain-specific LLMs
- Multimodal models
- Self-assessment systems
- Self-optimizing pipelines

Conclusion

You now have a comprehensive overview of modern predictive analytics: from the fundamentals to intelligent agents that use LLMs to automate the entire model lifecycle. This book serves as a solid foundation for building real, robust, and scalable solutions.

CHAPTER 1 — Fundamentals of Predictive Analytics

Predictive analytics is one of the cornerstones of modern intelligence. It allows businesses, governments, and individuals to anticipate behaviors, identify patterns, and make evidence-based decisions. Before we delve into intelligent agents and LLMs, we need to build a solid foundation—and that's what this chapter does.

1.1 What is Predictive Analytics?

Predictive analytics is the process of using historical data to predict future events. It combines:

- **Statistic**
- **Machine Learning**
- **Probabilistic models**
- **Data engineering**
- **Language Models (LLMs)**



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

The central idea is simple:

If we understand how something behaved in the past, we can estimate how it will behave in the future.

But execution is complex — it involves dirty data, flawed models, uncertainties, and human decisions.

1.2 Why Predictive Analytics has become so important

We live in a world where everything generates data:

- Shopping
- Clicks
- Sensors
- Conversations
- Images
- System logs
- Social media

The ability to transform this ocean of data into useful predictions is a huge competitive advantage.

Companies use predictive analytics to:

- Predicting demand
- Reduce churn
- Detect fraud
- Optimize prices
- Anticipating machine failures
- Personalize experiences

And now, with LLMs, that capability expands to unstructured data — text, audio, documents, code.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

1.3 Types of Predictive Problems

1.3.1 Regression

Prediction of ongoing values.

Examples:

- Property prices
- Future sales
- Temperature

1.3.2 Classification

Category forecast.

Examples:

- Will the customer cancel? (yes/no)
- Is email spam?
- Is this transaction fraudulent?

1.3.3 Time Series

Weather-based forecast.

Examples:

- Daily demand
- Stock price
- Energy consumption



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

1.3.4 Generative Models (LLMs)

LLMs are not traditional predictive models, but **they predict the next word** —and that makes them extremely powerful for:

- Create code
- Generate features
- Explain models
- Automate pipelines
- Integrate complex data

1.4 The Predictive Analytics Lifecycle

Predictive analytics is not just about training models. It's a complete process:

1. **Data collection**
2. **Cleaning and preparation**
3. **Feature engineering**
4. **Model training**
5. **Assessment**
6. **Implantation**
7. **Monitoring and retraining**

LLMs and intelligent agents can operate at **all** these stages.

1.5 The Role of LLMs in Modern Predictive Analytics

LLMs have changed the game. They can:

- Create templates automatically
- Explain results to humans
- Generate Python code on demand.
- Create features from text.
- Read documents and extract insights.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Orchestrating pipelines as autonomous agents

They don't replace traditional models — **they enhance them** .

1.6 Simple example: prediction with Python

To close the chapter, a quick and didactic example.

Loading data and training a linear regression.

```
python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Carregar dados
df = pd.read_csv("vendas.csv")

# Separar features e target
X = df[["preco", "estoque", "marketing"]]
y = df["vendas"]

# Dividir em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

# Treinar modelo
model = LinearRegression()
model.fit(X_train, y_train)

print("Coeficientes:", model.coef_)
print("Intercepto:", model.intercept_)
```

Simple, straightforward, and powerful — the essence of predictive analytics.

1.7 What comes next

In the next chapter, we will delve into **data preparation** , the most laborious and critical step in the entire process. Without good data, there is no good model.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

CHAPTER 2 — Data Preparation and Processing

Data preparation is the heart of predictive analytics. Sophisticated models don't compensate for bad data. This is where data scientists spend most of their time: cleaning, transforming, enriching, and understanding the data.

In this chapter, we will explore:

- Collect
- Cleaning
- Handling missing values
- Outlier detection
- Feature engineering
- Normalization and standardization
- Data division
- Automated preparation with agents and LLMs

2.1 The Importance of Data Preparation

Data rarely arrives ready-made. It comes:

- Incomplete
- Duplicates
- Inconsistent
- Full of noise
- In different formats

The quality of the data determines the quality of the predictions. A model is only as good as the data that feeds it.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

2.2 Data Collection

The data can come from:

- SQL Databases
- APIs
- CSV/Excel files
- Logs
- IoT Sensors
- PDFs and documents (where LLMs shine)
- Web scraping

Simple example: loading data

```
python
import pandas as pd

df = pd.read_csv("clientes.csv")
df.head()
```

2.3 Data Cleansing

2.3.1 Removal of duplicates

```
python
df = df.drop_duplicates()
```

2.3.2 Handling missing values

There are several strategies:

- Remove lines
- Fill in with mean/median
- Fill in with the most frequent value.
- Fill in with predictive models
- Using LLMs to infer textual values



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Example:

```
python
df["idade"] = df["idade"].fillna(df["idade"].median())
```

2.4 Detection and Treatment of Outliers

Outliers can distort models, especially regressions.

2.4.1 Using the IQR method

```
python
Q1 = df["rent"].quantile(0.25)
Q3 = df["rent"].quantile(0.75)
IQR = Q3 - Q1

filter = (df["income"] >= Q1 - 1.5*IQR) & (df["income"] <= Q3 + 1.5*IQR)
df_filtered = df[filter]
```

2.5 Data Transformation

2.5.1 Standardization (0 to 1)

```
python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df_norm = scaler.fit_transform(df[["idade", "renda"]])
```

2.5.2 Standardization (mean 0, standard deviation 1)

```
python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_std = scaler.fit_transform(df[["idade", "aluguel"]])
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

2.6 Resource Engineering

Creating good resources is more important than choosing the model.

2.6.1 Classic examples

- Convert dates to:
 - day of the week
 - month
 - holiday
 - seasonality
- Creating reasons:
 - per capita income
 - expense_per_visit
- Create binary variables (one-hot encoding)

```
python
df = pd.get_dummies(df, columns=["city"])
```

2.6.2 Resource Engineering with LLMs

LLMs can:

- Suggest new features
- Create semantic descriptions
- Extracting text entities
- Transforming documents into embeddings

Example of a prompt:

Code

```
"Analyze this dataset and suggest 15 new useful features for  
predicting churn."
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

2.7 Data Division

Separating training and test data prevents overfitting.

```
python
from sklearn.model_selection import train_test_split

X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)
```

2.8 Preprocessing Pipelines

Pipelines ensure consistency.

```
python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipeline.fit(X_train, y_train)
```

2.9 Automated Preparation with Agents and LLMs

Intelligent agents can:

- Detecting problematic columns
- Suggest changes
- Generate Python code automatically
- Create complete pipelines



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Validate data quality.

Example of an agent that analyzes a dataset.

Typical prompt:

Code

```
"Analyze the clients.csv file and generate a complete data quality report, including suggestions for data cleanup and feature engineering."
```

The agent can:

- Identify columns with many null values.
- Detect outliers
- Suggest encoding
- Creating ready-to-use code

2.10 Chapter Conclusion

Data preparation is the foundation of any predictive analytics project. Sophisticated models don't compensate for poorly handled data. By combining traditional techniques with the power of LLMs, you can dramatically accelerate and improve this step.

CHAPTER 3 — Traditional Predictive Models

Before we talk about intelligent agents and LLMs, we need to master classic machine learning models. They remain essential, fast, interpretable, and extremely efficient in many scenarios.

In this chapter, you will see:

- How do the main predictive models work?
- When to use each type
- Practical examples in Python
- Good modeling practices



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- How can LLMs help in this step?

3.1 What is a Predictive Model?

A predictive model learns patterns from historical data to predict future outcomes. It receives:

- **Features** (input variables)
- **Target** (variable to be predicted)

And learn a function $f(x)$ that approximates the real behavior.

3.2 Linear Regression

Linear regression is one of the oldest models and still one of the most useful.

When to use

- Prediction of continuous values
- Approximately linear relationships
- Interpretable problems

Example in Python

```
python
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("sales.csv")

X = df[["price", "stock", "marketing"]]
y = df["sales"]

model = LinearRegression()
model.fit(X, y)

print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

3.3 Logistic Regression

Despite the name, it is used for **binary classification** .

When to use

- Churn (yes/no)
- Fraud (fraud/non-fraud)
- Credit approval

Example

```
python
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression()
clf.fit(X_train, y_train)

pred = clf.predict(X_test)
```

3.4 Decision Trees

Rule-based models that are easy to interpret.

Advantages

- Interpretability
- Handles categorical data well.
- It does not require standardization.

Example

```
python
from sklearn.tree import DecisionTreeClassifier
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
tree = DecisionTreeClassifier(max_depth=5)
tree.fit(X_train, y_train)
```

3.5 Random Forest

An ensemble of multiple trees. More robust, more precise.

When to use

- Tabular data
- Problems with too many features
- When interpretability is not a priority.

Example

```
python
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=200)
rf.fit(X_train, y_train)
```

3.6 Gradient Boosting (XGBoost, LightGBM, CatBoost)

The most commonly used models in competitions and real-world applications.

Why

- High performance
- Handles tabular data well.
- Great for medium-sized datasets.

Example with XGBoost

```
python
from xgboost import XGBClassifier
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
xgb = XGBClassifier(  
    n_estimators=300,  
    learning_rate=0.05,  
    max_depth=6  
)  
  
xgb.fit(X_train, y_train)
```

3.7 KNN (K - Nearest Neighbors)

Based on proximity.

Disadvantages

- Slow on large datasets.
- Sensitive to scale

Example

```
python  
from sklearn.neighbors import KNeighborsClassifier  
  
knn = KNeighborsClassifier(n_neighbors=5)  
knn.fit(X_train, y_train)
```

3.8 SVM (Support Vector Machines)

Excellent for well-defined margins.

When to use

- Few features
- Well-separated classes
- Complex problems with the kernel





Voluntary Contribution

<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
 WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python
Practice with MYSQL on **AWS** | **Azure** | **GCP**

Example

```
python
from sklearn.svm import SVC

svm = SVC(kernel="rbf")
svm.fit(X_train, y_train)
```

3.9 Classical Neural Networks (MLP)

Before deep web networks, there were already MLPs (Multiple Landing Platforms).

Example

```
python
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(hidden_layer_sizes=(64, 32))
mlp.fit(X_train, y_train)
```

3.10 Comparing Models

Each model has its strengths:

Model	Best for	Strengths
Linear Regression	Simple relationships	Interpretation
Logistics	Binary classification	Simple and effective
Trees	Tabular data	Interpretable
Random Forest	General	Robust
XGBoost	Competition/production	High performance
SVM	Clear margins	High precision
KNN	Small datasets	Simplicity
MLP	Complex patterns	Flexible



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

3.11 How LLMs help in this step

LLMs can:

- Suggest the best model for the problem.
- Generate complete training code
- Explain hyperparameters
- Create automated pipelines
- Compare models and suggest improvements.
- Interpret results for humans

Example of a prompt:

Code

Analyze this dataset and suggest the 3 best models to predict churn, with justification.

3.12 Complete Example: Training and Evaluating Models

python

```
from sklearn.metrics import accuracy_score

models = {
    "RandomForest": RandomForestClassifier(),
    "XGBoost": XGBClassifier(),
    "Logistica": LogisticRegression(max_iter=1000)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    print(name, "->", acc)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

3.13 Chapter Conclusion

Traditional models remain the backbone of predictive analytics. They are fast, efficient, and interpretable—and combined with LLMs, they become even more powerful.

In the next chapter, we will delve into the world of **LLMs**, understand how they work, and how they can be integrated into the predictive pipeline.

CHAPTER 4 — Introduction to LLMs (Large Language Models)

Large-Scale Language Models — or simply **LLMs** — are neural networks trained on massive amounts of text to learn patterns, semantic relationships, linguistic structures, and even ways of reasoning. They don't just complete sentences: **they plan, explain, analyze, generate code, and solve complex problems**.

In this chapter, you will understand:

- What are LLMs?
- How they work
- Why are they so powerful?
- How to apply them in predictive analytics
- How to integrate them with Python
- How to use them as intelligent agents



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

4.1 What is a Language Model?

An LLM is a model trained to predict the next word in a sequence. This simple task, when scaled to billions of parameters and trillions of tokens, results in emergent capabilities:

- Reasoning
- Planning
- Summary
- Translation
- Code generation
- Data analysis
- Creating machine learning pipelines

They have become a kind of "cognitive engine" for intelligent systems.

4.2 How LLMs Work (Intuitive View)

Without getting into heavy mathematics, the operation can be summarized as follows:

1. **Tokenization:** The text is divided into smaller units (tokens).
2. **Embeddings:** Each token is transformed into a numeric vector.
3. **Transformer Architecture:** The model uses attention mechanisms to understand relationships between tokens.
4. **Massive training:** The model reads trillions of words and learns patterns.
5. **Inference:** Given a prompt, it generates the next most likely word — repeatedly.

This sequence generates coherent texts, functional code, and even complex analyses.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

4.3 Emerging Capabilities

LLMs demonstrate skills that were not explicitly programmed:

- Solve math problems
- Create action plans
- Explaining complex concepts
- Write functional Python code.
- Analyze datasets
- Create machine learning models
- Acting as autonomous agents

These capabilities emerge from the size and diversity of the training.

4.4 Why LLMs are important for Predictive Analytics

LLMs don't replace traditional models — they **amplify** the process.

They can:

- Create features automatically.
- Explaining models to stakeholders
- Generate preprocessing code
- Create complete ML pipelines.
- Interpreting unstructured data
- Automate experiments
- Create agents that train, evaluate, and implement models.

Example of use:

Code

```
"Analyze this dataset and suggest the 5 best models to predict churn."
```

4.5 LLMs as Data Engineering Tools

LLMs can:



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Detect inconsistencies
- Suggest changes
- Create cleanup scripts
- Generate SQL
- Create automatic documentation.

Example:

Code

```
Write a Python script to clean this dataset and remove outliers.
```

4.6 LLMs as Machine Learning Tools

They can:

- Create templates
- Adjust hyperparameters
- Compare algorithms
- Explain metrics
- Create executive reports.

Example:

Code

```
"Train a Random Forest model to predict sales and explain the result."
```

4.7 LLMs as Intelligent Agents

This is where the magic begins.

An agent based on LLM can:

- To receive a task.
- Plan the steps
- Execute code
- Read results



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Adjust strategies
- Repeat the cycle until the goal is achieved.

This transforms the LLM into an **autonomous data scientist** .

4.8 Integration with Python

4.8.1 Simple example: using an LLM to generate code

```
python
from openai import OpenAI

client = OpenAI()

prompt = """
Write a Python code that loads a CSV file, removes null values, and
performs linear regression.
"""

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}]
)

print(response.choices[0].message["content"])
```

4.8.2 Using LLMs to analyze data

```
python
prompt = f"""
Here are the first few rows of the dataset:

{df.head().to_string()}

Explain what you observe and suggest features.
"""
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

4.9 LLMs and Unstructured Data

LLMs shine in:

- Text
- PDFs
- Emails
- Logs
- Conversations
- Legal documents
- Customer reviews

They can transform all of this into:

- Features
- Insights
- Abstracts
- Classifications
- Embeddings

4.10 Embeddings and Vectorization

Embeddings are numerical representations of text. They allow:

- Semantic search
- Clustering
- Recommendation
- Similarity
- Textual classification



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Example

```
python
from openai import OpenAI

client = OpenAI()

emb = client.embeddings.create(
    model="text-embedding-3-small",
    input="customer complained about the delivery delay"
)

vector = emb.data[0].embedding
```

4.11 Limitations of LLMs

Despite their power, they have limitations:

- They may hallucinate.
- They don't have direct access to the internet (unless integrated).
- They are not a replacement for experts.
- They may generate incorrect code.
- They need human validation.

Therefore, intelligent agents generally combine:

- LLMs
- External tools
- Code execution
- Safety rules

4.12 Chapter Conclusion

LLMs are the foundation of the next generation of intelligent systems. They not only understand language—they **reason, plan, and execute** .

Combined with Python and machine learning techniques, they become powerful tools for predictive analytics and automation.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

In the next chapter, we'll delve into the most exciting topic in the book:

👉 **Intelligent Agents** — how to create autonomous systems that use LLMs to predict, decide, and act.

CHAPTER 5 — Intelligent Agents

Intelligent agents are systems capable of **perceiving their environment, making decisions, and executing actions** to achieve objectives. When we combine agents with LLMs (Lead-Lead Systems), we create systems that:

- They plan
- They write code.
- They perform tasks.
- They evaluate the results.
- They adjust their strategies.
- They learn from the process.

It's like having a "freelance data scientist" working alongside you.

5.1 What is an Agent?

An agent is defined by three elements:

1. **Perception** — receives information from the environment.
2. **Decision** — choose the best course of action.
3. **Action** — performs something in the environment.

In predictive analytics, the environment can be:

- A dataset
- A database
- An ML pipeline
- A set of APIs
- A file system



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- A Python runtime environment

5.2 Types of Agents

5.2.1 Reactive Agents

They respond directly to stimuli. Example: a bot that answers questions.

5.2.2 Goal-Based Agents

They plan actions to achieve goals. Example: "optimize this model until it reaches 90% accuracy".

5.2.3 Utility-Based Agents

They choose actions that maximize an expected value. Example: recommendation systems.

5.2.4 Independent Agents with LLMs

The most advanced. They:

- They plan
- They execute code.
- They evaluate the results.
- They adjust the plan.
- They repeat the cycle.

5.3 Why Agents Are Important in Predictive Analytics

Agents enable:

- Automating entire pipelines
- Create models on demand.
- Monitor drift



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Automatically retrain models
- Generate executive reports
- Integrate structured and unstructured data
- Create systems that adapt.

Imagine asking:

"Create a churn model, evaluate it, improve it, and generate a final report."

And the agent does everything.

5.4 Architecture of an Agent with LLM

A modern agent typically has:

1. **LLM** — brain
2. **Tools** — possible actions
3. **Memory** — persistent context
4. **Environment** — where he operates
5. **Reasoning loop** — think → act → observe → adjust

5.5 Tools

Tools are external functions that the agent can call, such as:

- Execute Python code
- Read files
- Training models
- Consult APIs
- Accessing databases
- Generate graphs

The LLM decides **when** and **how** to use each tool.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

5.6 Popular Frameworks for Agents

✓ LangChain

The most well-known. Allows you to create agents with complex tools, memory, and workflows.

✓ AutoGen

Focused on collaborative agents (multiple agents communicating with each other).

✓ CrewAI

Designed for teams of agents with defined roles.

✓ LlamaIndex

Excellent for agents who need access to documents.

5.7 Creating a Simple Agent with Python (Language Chain)

Here is a didactic example of an agent that can train models:

```
python
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI

def train_model(path):
    import pandas as pd
    from sklearn.linear_model import LinearRegression

    df = pd.read_csv(path)
    X = df.drop("target", axis=1)
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
y = df["target"]

model = LinearRegression()
model.fit(X, y)

Return "Model trained successfully!"

tools = [
    Tool(
        name="TrainModel",
        func=train_model,
        description="Trains a predictive model using a CSV"
    )
]

llm = OpenAI()
agent = initialize_agent(tools, llm, agent="zero-shot-react-
description")

agent.run("Train a model using the data.csv file")
```

This agent:

- Receive the instruction
- Decide to use the tool
- Execute the code.
- Returns the result.

5.8 Multi-step Agents (Chain-of-Thought)

Agents can:

- Plan
- To execute
- To assess
- Replan

Example of instruction:

"Optimize this model until it achieves at least 85% accuracy."



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

The agent can:

1. Train model
2. To assess
3. Adjust hyperparameters
4. Repeat
5. Generate report

5.9 Agents for ML Pipeline Automation

Agents can automate:

- Data cleansing
- Feature engineering
- Model selection
- Hyperparameter tuning
- Assessment
- Implantation
- Monitoring

This creates a **smart AutoML** , much more flexible than traditional tools.

5.10 Agents for Monitoring and Retraining

An agent can:

- Detect drift
- Comparing distributions
- retraining models
- Generate alerts
- Log entries

Example of a function used by agents:

```
python
from scipy.stats import ks_2samp
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
def detect_drift(df_new, df_training):  
    results = {}  
    for col in df_treino.columns:  
        results[col] = ks_2samp(df_train[col], df_new[col]).pvalue  
    return results
```

5.11 Collaborative Agents

Several agents can work together:

- An agent analyzes data.
- Another trains models.
- Another writes reports.
- Another validates results.

It's like an automated team of data scientists.

5.12 Limitations of the Agents

Despite their power, lawmakers face challenges:

- They can make mistakes.
- They may generate incorrect code.
- They need human validation.
- They can get into loops.
- They depend on the quality of the tools.

Therefore, agents must be monitored.

5.13 Chapter Conclusion

Intelligent agents are the natural evolution of predictive analytics. They allow the creation of systems that:

- They understand the objectives.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- They plan actions.
- They execute code.
- They learn from results.
- They automate complex processes.

In the next chapter, we'll see how to use agents and LLMs to **create intelligent features**, one of the most important steps in modeling.

CHAPTER 6 — Building a Predictive Agent with Python

So far, you have learned:

- How to prepare data
- How to train traditional models
- How LLMs work
- What are intelligent agents?

Now let's put all this together to create a **predictive agent** capable of:

- Read data
- Create features
- Training models
- Evaluate results
- Adjust hyperparameters
- Generate reports
- Repeat the cycle automatically

This is the first step in building an **autonomous data scientist**.

6.1 What is a Predictive Agent?

A predictive agent is a system that:

1. It receives a goal (e.g., "predict churn").
2. Analyze the available data.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

3. Decide which model to use.
4. Train and evaluate the model.
5. Adjusts hyperparameters.
6. Generates a final report.

It matches:

- **LLM** as brain
- **Python tools** like arms
- **Memory** as context
- **Environment** as a space for execution.

6.2 Basic Architecture of a Predictive Agent

A modern predictive agent typically has:

- **LLM** for Reasoning and Planning
- **Tools** for running code
- **Python functions** for manipulating data
- **ML pipeline** for training models
- **Logic loop** to iterate until improvement

Visually:

Code

Objective → Planning → Execution → Evaluation → Adjustment → Result

6.3 Creating Tools for the Agent

Tools are Python functions that the agent can call.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Example: tool for training a model

```
python
def train_model(csv_path):
    import pandas as pd
    from sklearn.model_selection import train_test_split
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.metrics import accuracy_score

    df = pd.read_csv(csv_path)

    X = df.drop("target", axis=1)
    y = df["target"]

    X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2)

    model = RandomForestClassifier(n_estimators=200)
    model.fit(X_train, y_train)

    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)

    return f"Accuracy: {acc:.4f}"
```

6.4 Creating the Agent with LangChain

Here is a simple agent that uses the tool above.

```
python
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI

tools = [
    Tool(
        name="TrainModel",
        func=train_model,
        description="Trains a predictive model using a CSV with a 'target' column"
    )
]
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
]

llm = OpenAI()

agent = initialize_agent(
    tools,
    llm,
    agent="zero-shot-react-description"
)

agent.run("Train a model using the data.csv file")
```

This agent:

- Read the instruction.
- Decide to use the tool
- Execute the code.
- Returns accuracy

6.5 Creating a Multi-Step Agent (Smart AutoML)

Now let's create an agent that:

- Analyze the data.
- Suggests features
- Trains various models.
- Compare results
- Choose the best one.

Tool for testing various models

```
python
def compare_models(csv_path):
    import pandas as pd
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.linear_model import LogisticRegression
    from xgboost import XGBClassifier
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
df = pd.read_csv(csv_path)

X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

models = {
    "RandomForest": RandomForestClassifier(),
    "Logistica": LogisticRegression(max_iter=1000),
    "XGBoost": XGBClassifier()
}

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    results[name] = acc

return results
```

6.6 Agent that Analyzes Data with LLM

The agent can use the LLM to:

- Identify problematic columns
- Suggest changes
- Create features
- Explain patterns

Example of a prompt

Code

Analyze the dataset below and suggest 10 new useful features for predicting the target:

```
{df.head().to_string() }
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

6.7 Complete Agent: Plan → Execute → Evaluate

A complete agent can:

1. Read the dataset
2. Analyze columns
3. Create features
4. Testing models
5. Adjust hyperparameters
6. Generate final report

Typical flow

Code

User: "Create the best possible model to predict churn."

Agent:

- Analyzes data
- Suggests features
- Trains models
- Compare results
- Adjust hyperparameters
- Generates final report

6.8 Example of an Agent with an Improvement Loop

python

```
def improve_model(csv_path, iterations=3):
    best_acc = 0
    best_model = None

    for i in range(iteracoes):
        result = compare_models(csv_path)
        name = max(result, key=result.get)
        acc = result[name]

        if acc > melhor_acc:
            best_acc = acc
            best_model = name
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
return f"Best model: {best_model} with accuracy {best_accuracy:.4f}"
```

6.9 How LLMs Help in the Process

LLMs can:

- Explain why one model was better.
- Suggest hyperparameters
- Create executive reports.
- Generate insights from the data.
- Create Python code on demand.
- Orchestrating the entire pipeline

6.10 Chapter Conclusion

In this chapter, you learned how to:

- Create Python tools for agents.
- Integrating LLMs with ML pipelines
- Create agents that train and evaluate models.
- Build intelligent AutoML
- Automating complex predictive analytics tasks

In the next chapter, we'll explore how to use LLMs and agents to **create intelligent features**, one of the most important steps in any model.

CHAPTER 7 — LLMs for Feature Engineering

Feature engineering is the art of transforming raw data into useful variables for predictive models. This is where human intelligence has always made the biggest difference—until now.

With LLMs and intelligent agents, this step becomes:

- faster



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- more creative
- more automated
- more effective

In this chapter, you will learn:

- What is feature engineering?
- Traditional techniques
- How LLMs extend this step
- How to create features from text, dates, categories, and documents.
- How can agents automate the entire process?
- Practical examples in Python

7.1 What is Feature Engineering?

Feature engineering is the process of:

- To create
- Transform
- Select
- To combine

variables that help the model learn patterns.

A good set of features can transform a simple model into a powerful one.

7.2 Why Feature Engineering is so important

Models learn from features. If the features are bad, the model will be bad—even if it is sophisticated.

Good features:

- increase accuracy
- reduce overfitting
- They make the model more interpretable.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python

Practice with MYSQL on  AWS |  Azure |  GCP

- They accelerate the training.

7.3 Traditional Feature Engineering Techniques

✓ 7.3.1 Mathematical Transformations

- log
- source
- square
- normalization

✓ 7.3.2 Category Coding

- One-hot encoding
- Label encoding
- Target encoding

✓ 7.3.3 Temporal Features

- day of the week
- month
- seasonality
- holidays

✓ 7.3.4 Interactions

- multiplications
- divisions
- nonlinear combinations

✓ 7.3.5 Dimensionality Reduction

- PCA
- UMAP



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

7.4 Where LLMs fit into the game

LLMs achieve:

- Read the dataset
- Understanding the context
- Suggest relevant features
- Creating Python code to implement them.
- Generate features from text
- Create embeddings
- Explain why a feature is useful.

Example of a prompt:

Code

```
Analyze this dataset and suggest 15 new useful features for predicting churn.
```

7.5 Resource Engineering with LLMs: Practical Examples

7.5.1 Creating features from text

python

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=500)
X_text = vectorizer.fit_transform(df["commentarios"])
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

7.5.2 Creating embeddings with LLMs

Embeddings transform text into numeric vectors.

```
python
from openai import OpenAI

client = OpenAI()

emb = client.embeddings.create(
    model="text-embedding-3-small",
    input=df["comments"].tolist()
)

vectors = [e.embedding for e in emb.data]
```

These vectors can be used in:

- classification models
- clustering
- recommendation

7.6 Resource Engineering for Structured Data

LLMs may suggest:

- reasons
- differences
- indicators
- unexpected combinations

Example of a prompt:



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Code

Suggest 10 new features for a credit dataset with the following columns: age, income, debts, score.

7.7 Resource Engineering for Time Series

LLMs may suggest:

- movable windows
- moving averages
- lags
- seasonality
- decomposition

Example in Python:

```
python
df["lag_1"] = df["sales"].shift(1)
df["media_7"] = df["sales"].rolling(7).mean()
```

7.8 Resource Engineering with Intelligent Agents

Agents can:

- analyze the dataset
- suggest features
- generate code
- execute the code
- validate results
- iterate until it improves

Typical flow:

Code

```
User: "Improve the performance of this model by creating new features."
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Agent:
- Analyze the data
Suggests features
- Implements
- Train model
- Evaluate
Repeat

7.9 Example of an Agent that Creates Features

```
python
def create_features(path_csv):
    import pandas as pd

    df = pd.read_csv(csv_path)

    df["aluguel_per_capita"] = df["aluguel"] / df["num_moradores"]
    df["rent_debt"] = df["debts"] / df["rent"]
    df["log_age"] = np.log(df["age"] + 1)

    df.to_csv("dados_com_features.csv", index=False)
    Return "New features created!"
```

The agent can call this tool automatically.

7.10 Feature Selection with LLMs

LLMs can:

- Explain which features are most important.
- suggest removing redundant columns
- identify collinearity
- suggest PCA or other techniques

Example of a prompt:

Code

Which features in this dataset are redundant and should be removed?



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

7.11 Feature Engineering for Unstructured Data

LLMs shine in:

- text
- PDFs
- emails
- logs
- legal documents
- customer reviews

They can:

- extract entities
- classifying feelings
- generate summaries
- create embeddings
- transform text into numeric features

7.12 Chapter Conclusion

Feature engineering is one of the most important steps in predictive analytics — and now, with LLMs and agents, it becomes even more so.

- more creative
- more automated
- most powerful

You learned:

- Traditional techniques



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- How LLMs expand the process
- How to create features from text, dates, and time series.
- How can agents automate everything?

In the next chapter, we'll delve into **time series with agents** , where automation truly shines.

CHAPTER 8 — Time Series with Agents and LLMs

Time series are sets of data ordered in time. Unlike traditional models, here the order matters—a lot.

By combining classic techniques with intelligent agents, we were able to:

- Create temporary features automatically.
- Adjusting models such as Prophet, ARIMA, LSTM
- Detect anomalies
- Automate predictions
- Create autonomous forecasting pipelines

This chapter shows how to do this in practice.

8.1 What is a Time Series?

A time series is a sequence of observations indexed in time.

Examples:

- Daily sales
- Temperature per hour
- Stock price
- Energy consumption
- Network traffic
- Failure occurrences



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Key features:

- Trend
- Seasonality
- Cycles
- Noise
- External events

8.2 Main Forecasting Techniques

✓ Statistical Models

- ARIMA
- SARIMA
- Holt-Winters

✓ ML Based Models

- Random Forest
- XGBoost
- Regression with lags

✓ Modern Designs

- Prophet
- LSTMs
- Transformers for time series

✓ Hybrid Models with LLMs

- LLM creates features



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- ML makes the prediction.
- The agent coordinates everything.

8.3 Temporal Data Preparation

Creating lags

```
python
df["lag_1"] = df["sales"].shift(1)
df["lag_7"] = df["sales"].shift(7)
```

Moving averages

```
python
df["media_7"] = df["sales"].rolling(7).mean()
```

Data features

```
python
df["day_week"] = df["date"].dt.dayofweek
df["month"] = df["date"].dt.month
df["holiday"] = df["date"].isin(holiday_list).astype(int)
```

8.4 Feature Engineering with LLMs for Time Series

LLMs may suggest:

- movable windows
- trend decomposition
- seasonal indicators
- relevant external events
- unexpected combinations

Example of a prompt:

Code

```
Suggest 20 temporal features to improve weekly sales forecasting.
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

8.5 Using Prophet for Forecasting

Prophet is simple and powerful.

```
python
from prophet import Prophet

df_prophet = df.rename(columns={"data": "ds", "sales": "y"})

model = Prophet()
model.fit(df_prophet)

future = model.make_future_dataframe(periods=30)
forecast = model.predict(future)
```

8.6 Creating an Agent to Adjust the Prophet

An agent can:

- test seasonality
- adjust hyperparameters
- compare metrics
- generate graphs
- explain results

Typical flow:

Code

```
User: "Optimize Prophet to predict sales."
Agent:
- Tests for daily, weekly, and yearly seasonality.
Adjust changepoints
- Assess RMSE
Choose the best one.
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

8.7 Forecasting with ARIMA

```
python
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(df["sales"], order=(5,1,0))
result = model.fit()

forecast = result.forecast(steps=30)
```

Agents can automatically test:

- p
- d
- q
- seasonality

8.8 Deep Learning Models for Time Series

Simple LSTM

```
python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

model = Sequential([
    LSTM(64, input_shape=(window, 1)),
    Dense(1)
])

model.compile(optimizer="adam", loss="mse")
model.fit(X_train, y_train, epochs=20)
```

Agents can:



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- adjust number of neurons
- test windows
- compare architectures

8.9 Agent for Complete (End-to-End) Prediction

A time series predictive agent can:

1. Analyze the dataset
2. Create temporary features
3. Test Prophet, ARIMA, XGBoost and LSTM
4. Compare metrics (RMSE, MAPE)
5. Choosing the best model
6. Generate executive report
7. Create a forecast chart
8. Explain anomalies

Example of a tool used by the agent

```
python
def evaluate_temporal_models(df):
    from prophet import Prophet
    from sklearn.metrics import mean_absolute_error

    df_prophet = df.rename(columns={"data": "ds", "sales": "y"})
    model = Prophet()
    model.fit(df_prophet)

    future = model.make_future_dataframe(periods=30)
    forecast = model.predict(future)

    mae = mean_absolute_error(df_prophet["y"],
                             forecast["yhat"][:len(df_prophet)])
    return f"MAE of the Prophet: {mae:.4f}"
```

8.10 Anomaly Detection with Agents

Agents can:



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- detect spikes
- identify unexpected falls
- correlate external events
- generate explanations

Simple example

```
python
df["zscore"] = (df["sales"] - df["sales"].mean()) / df["sales"].std()
anomalies = df[df["zscore"].abs() > 3]
```

The agent can explain:

"The drop in sales on March 12th coincides with a failure in the payment system."

8.11 How LLMs Help in Time Series

LLMs can:

- suggest models
- explain seasonality
- interpreting graphs
- generate executive reports
- create features
- automate experiments
- identify relevant external events

Example:

Code

```
Explain why there was a sales spike on December 15th.
```

8.12 Chapter Conclusion

In this chapter, you learned:



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- Fundamentals of time series
- Classic and modern techniques
- How to use Prophet, ARIMA and LSTM
- How to create temporary features
- How agents and LLMs automate forecasting
- How to detect anomalies
- How to build autonomous pipelines

CHAPTER 9 — Evaluation of Predictive Models

Training a model is easy. Evaluating it correctly is difficult—and absolutely essential.

The assessment ensures that:

- the model generalizes
- He is not decorating the data.
- It works in new scenarios.
- It is robust to noise.
- It is fair and ethical.
- meets business requirements

In this chapter, you will learn:

- How to evaluate regression, classification, and time series models.
- How to avoid common pitfalls
- How to use metrics correctly
- How agents and LLMs help in the assessment
- How to create automated reports
- How to monitor models in production

9.1 Why Evaluating Models is so Important

Without proper assessment, you may:

- choosing the wrong model



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- overestimate performance
- deploying something that fails in the real world
- making bad decisions
- lose money

Evaluation is the **safety brake** of predictive analytics.

9.2 Data Division: Training, Validation, and Testing

✓ Training

The model learns.

✓ Validation

Used to adjust hyperparameters.

✓ Test

Used to measure real-world performance.

Example:

```
python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

9.3 Evaluation of Classification Models

✓ 9.3.1 Accuracy

Percentage of correct answers.



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
python
from sklearn.metrics import accuracy_score
accuracy_score(y_test, pred)
```

Good for balanced classes.

✅ 9.3.2 Accuracy, Recall and F1-score

Essential for:

- fraud
- churn
- disease detection
- unbalanced ranking

```
python
from sklearn.metrics import classification_report
print(classification_report(y_test, pred))
```

✅ 9.3.3 Confusion Matrix

Shows errors by class.

```
python
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, pred)
```

✅ 9.3.4 AUC-ROC

It measures the separability between classes.

```
python
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test, probas)
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

9.4 Evaluation of Regression Models

✓ 9.4.1 MAE (Mean Absolute Error)

Mean absolute error.

```
python
from sklearn.metrics import mean_absolute_error
mean_absolute_error(y_test, pred)
```

✓ 9.4.2 RMSE (Root Mean Squared Error)

It penalizes major mistakes.

```
python
from sklearn.metrics import mean_squared_error
rmse = mean_squared_error(y_test, pred, squared=False)
```

✓ 9.4.3 R²

How much of the variability does the model explain?

```
python
from sklearn.metrics import r2_score
r2_score(y_test, pred)
```

9.5 Time Series Evaluation

Time series require specific metrics.

✓ MAP



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Average percentage of error.

```
python
from sklearn.metrics import mean_absolute_percentage_error
mean_absolute_percentage_error(y_test, pred)
```

✓ Temporal RMSE

Good for comparing models.

✓ Backtesting

It simulates predictions from the past.

9.6 Cross-Validation

It prevents overfitting and ensures robustness.

```
python
from sklearn.model_selection import cross_val_score
cross_val_score(model, X, y, cv=5)
```

9.7 Overfitting and Underfitting

✓ Overfitting

The model learns a lot from training.

Signs:

- very good workout
- bad test

✓ Underfitting



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Too simple a model.

Signs:

- bad training
- bad test

9.8 Comparing Models

Example:

```
python
models = {
    "RF": RandomForestClassifier(),
    "XGB": XGBClassifier(),
    "LR": LogisticRegression(max_iter=1000)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, accuracy_score(y_test, pred))
```

9.9 Evaluation with Agents and LLMs

Agents can:

- Generate automatic reports.
- explain metrics
- compare models
- suggest improvements
- detect overfitting
- recommend hyperparameters

Example of a prompt:

Code

```
Explain why the XGBoost model performed better than Random Forest.
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

9.10 Automated Reports with LLMs

LLMs can transform metrics into executive reports.

Example:

Code

Generate an executive report explaining the results below:

```
Accuracy: 0.87  
AUC: 0.92  
F1: 0.84
```

9.11 Monitoring Production Models

Evaluation doesn't end with testing. Models degrade over time.

What to monitor:

- data drift
- concept drift
- drop in performance
- changes in user behavior
- anomalies

Example of drift detection

```
python  
from scipy.stats import ks_2samp  
  
ks_2samp(df_treino["idade"], df_novo["idade"])
```



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

9.12 Chapter Conclusion

In this chapter, you learned:

- How to evaluate classification, regression, and time series models.
- How to avoid overfitting
- How to compare models
- How to use agents and LLMs for assessment
- How to generate automatic reports
- How to monitor models in production

Evaluation is what ensures that your model works **in the real world** , not just on a laptop.

CHAPTER 10 — Deployment and Monitoring with Intelligent Agents

Training a model is just the beginning. The real challenge is putting it into production and ensuring it continues to function over time.

Models degrade. Data changes. Users change. The environment changes.

Therefore, we need:

- deployment pipelines
- continuous monitoring
- drift detection
- automatic retraining
- intelligent agents to orchestrate everything



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

This chapter shows how to do this in practice.

10.1 What is Model Implementation?

Deploying a model means making it available for real-world use, whether that is:

- via API
- on a dashboard
- in an internal system
- in an app
- in an automated pipeline

Implementation involves:

- versioning
- execution environment
- scalability
- security
- monitoring

10.2 Ways to Deploy Models

✓ 10.2.1 REST APIs

The model is exposed as an endpoint.

✓ 10.2.2 Batch Processing

Batch forecasts (daily, weekly).

✓ 10.2.3 Streaming



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>
[WhatsApp](https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/) 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Real-time predictions (fraud, IoT).

✓ 10.2.4 Edge

Models running on local devices.

✓ 10.2.5 Intelligent Agents

The model is part of an autonomous system that:

- reads data
- generates forecasts
- makes decisions
- retraining when necessary

10.3 Creating a Simple API with FastAPI

```
python
from fastAPI import FastAPI
import joblib
import pandas as pd

app = FastAPI()
model = joblib.load("model.pkl")

@app.post("/prever")
def predict(data: dict):
    df = pd.DataFrame([data])
    pred = model.predict(df)[0]
    return {"prediction": int(pred)}
```

10.4 Model Monitoring

Monitoring is just as important as training.

✓ What to monitor:



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- performance (AUC, RMSE, F1)
- data drift
- concept drift
- volume of requests
- latency
- errors
- anomalies

10.5 Drift Detection

Drifting occurs when:

- The data changes.
- user behavior changes
- the environment changes

Example of drift detection with KS-test

```
python
from scipy.stats import ks_2samp

def detect_drift(df_training, df_new):
    results = {}
    for col in df_training.columns:
        p = ks_2samp(df_training[col], df_new[col]).pvalue
        results[col] = p
    return results
```

If $p < 0.05$, there is strong evidence of drift.

10.6 Automatic Retraining

When the model degrades, it must be retrained.

Typical flow:

1. Detect drift



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

2. Validate impact
3. Collect data
4. Retrain model
5. To assess
6. Deploy new version
7. Record history

Agents can automate all of this.

10.7 Intelligent Agents for Monitoring

An agent can:

- monitor metrics
- detect drift
- generate alerts
- retraining models
- compare versions
- decide when to implement

Flow:

Code

Agent:

1. Collect new data
2. Compare with training data
3. Detects drift
4. Retraining model
5. Evaluate
6. Deploy new version
7. Generate report

10.8 Creating a Monitoring Agent in Python

python

```
def monitoring_agent(training_path, new_path):
```



<https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/>

WhatsApp 55 21 999618643

Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
import pandas as pd

df_treino = pd.read_csv(caminho_treino)
df_novo = pd.read_csv(caminho_novo)

drift = detect_drift(df_train, df_new)

columns_with_drift = [c for c, p in drift.items() if p < 0.05]

if columns_with_drift:
    return f"Drift detected in columns: {columns_with_drift}"
else:
    Return "No drift detected."
```

10.9 Model Versioning

Each model should have:

- ID
- creation date
- metrics
- hyperparameters
- data used
- code version

Common tools:

- MLflow
- DVC
- Weights & Biases

10.10 Automated Reports with LLMs

LLMs can transform metrics into executive reports.

Example:

Code



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Generate an executive report explaining the model's performance decline between January and March.

The agent can:

- analyze metrics
- correlate events
- suggest actions

10.11 Full Automation with Agents

An autonomous pipeline can:

1. Read new data
2. Detect drift
3. Retrain model
4. To assess
5. Implement
6. Generate report
7. Notify stakeholders

This is the future of predictive analytics.

10.12 Chapter Conclusion

In this chapter, you learned:

- How to deploy models
- How to monitor performance
- How to detect drift
- How to automatically retrain
- How to use agents for automation



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- How to generate executive reports with LLMs

Now you have everything you need to put models into production in a robust and intelligent way.

CHAPTER 11 — Real-World Cases of Predictive Analytics with Agents and LLMs

Up to this point, you've learned techniques, models, agents, pipelines, and automation. Now it's time to see how all of this applies to **real-world situations** across different industries.

This chapter presents:

- Real cases
- Typical architectures
- How agents and LLMs scale each solution
- Practical examples
- Challenges and opportunities

11.1 Demand Forecasting

Context

Companies need to anticipate:

- sales
- stock
- consumption



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- traffic
- production

As it was before

Traditional models such as ARIMA, Prophet, and regressions.

How is it now with agents?

Agents can:

- collect data automatically
- create temporary features
- testing various models
- adjust hyperparameters
- generate daily forecasts
- detect anomalies
- retrain when necessary

Example of flow

Code

Agent:

1. Collect daily sales
2. Creates lags and moving averages.
3. Test Prophet, ARIMA and XGBoost
4. Choose the best model.
5. Generates a 30-day forecast.
6. Send report to the team.

11.2 Fraud Detection



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Context

Fraud occurs in:

- credit cards
- e-commerce
- insurance
- banks
- Telecom

Challenges

- unbalanced data
- patterns that change rapidly
- need for real-time decisions

How agents help

- monitor transactions
- detect anomalies
- adjust thresholds
- retrain models
- They explain decisions to auditors.

Example of a feature created by LLM

Code

"Create a feature that represents the frequency of purchases above R\$ 500 in the last 7 days."

11.3 Churn (Customer Cancellations)

Context



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Companies want to predict who will cancel:

- banks
- telecom
- SaaS
- gyms
- retail

How agents help

- They analyze behavior.
- They create features automatically.
- testing models
- They explain why the customer will cancel.
- suggest retention actions

Example of an explanation generated by LLM

Code

```
The customer presents a high risk of churn due to:  
- increase in complaints  
- decrease in product usage  
- late payments
```

11.4 Credit Risk

Context

Financial institutions need to anticipate:

- default
- credit score
- loan risk



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

How agents help

- analyze history
- create financial features
- adjust models
- explain decisions (compliance)
- monitor changes in behavior

Example of a feature created automatically

```
python
df["comprometimento_renda"] = df["valor_parcela"] / df["renda"]
```

11.5 Predictive Maintenance

Context

Industries want to predict failures in:

- machines
- engines
- sensors
- production lines

How agents help

- They analyze time series from sensors.
- detect anomalies
- They foresee failures.
- maintenance is recommended
- They generate automatic reports.

Example of an insight generated by LLM

Code



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

Engine vibration has increased by 32% in the last 48 hours, indicating possible bearing wear.

11.6 Text and Document Analysis

Context

Companies have tons of unstructured data:

- emails
- PDFs
- contracts
- complaints
- chats
- social media

How agents help

- extract entities
- classify feelings
- summarize documents
- create textual features
- integrate text with predictive models

Example

Turning complaints into features:



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

```
python
from sklearn.feature_extraction.text import TfidfVectorizer

X_text =
TfidfVectorizer(max_features=300).fit_transform(df["reclamacao"])
```

11.7 Product Recommendation

Context

Used in:

- e-commerce
- streaming
- delivery apps
- marketplaces

How agents help

- They analyze behavior.
- create embeddings
- they suggest products
- They explain the recommendations.
- They adjust models automatically.

11.8 Network Security and Monitoring

Context

Companies need to detect:



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- attacks
- suspicious accesses
- traffic anomalies

How agents help

- They analyze logs.
- They detect strange patterns.
- correlate events
- suggest mitigation actions

11.9 Predictive Marketing

Context

Predict:

- best time to send campaigns
- probability of purchase
- customer segmentation

How agents help

- create clusters
- They analyze behavior.
- they suggest customized campaigns
- automate A/B testing

11.10 Chapter Conclusion

In this chapter, you saw how predictive analytics with agents and LLMs applies to:

- retail
- finances
- industry
- telecom



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

- health
- marketing
- security
- logistics

The agents make everything easier:

- fast
- intelligent
- automated
- explainable
- scalable

CHAPTER 12 — The Future of Predictive Analytics

Predictive analytics has evolved from simple regressions to complex models, and now to intelligent agents using LLMs. But this journey is only just beginning. The future promises even more autonomous, multimodal, and integrated systems.

12.1 Emerging Trends

Fully autonomous agents

- Virtual data scientists who plan, train, evaluate, and deploy models without human intervention.
- Capable of continuously learning from new data.

Domain-specific LLMs



Predictive Analysis with Agents and LLMs in Action: Hands-On Python

Practice with MYSQL on  AWS |  Azure |  GCP

- Trained models for specific areas: healthcare, finance, law, engineering.
- More accurate and reliable in critical contexts.

✓ Multimodal models

- Integration of text, image, audio, video, and sensors.
- Richer and more contextual forecasts.
- Example: predicting machine failures using text from reports + sensor data + thermal images.

✓ Advanced explainability

- Models that not only predict, but explain in natural language.
- Automated reports for executives and specialists.
- Transparency as an ethical requirement.

✓ Intelligent AutoML

- Pipelines that optimize themselves.
- Agents who test, evaluate, and choose the best model without supervision.
- Integration with LLMs for reporting and insights.

12.2 The Role of Agents in the Future

Intelligent agents will be:

- **Orchestrators** of complex pipelines
- Data quality **guardians**
- Drift and anomaly **monitors**
- **Consultants** who explain results.
- **Collaborators** who work as part of a team with humans.

They don't replace data scientists — but they expand their capabilities.

12.3 Ethics and Responsibility



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

With great power comes great responsibility. The future of predictive analytics demands attention to:

- **Privacy**
- **Algorithmic bias**
- **Transparency**
- **Security**
- **Social impact**

Agents and LLMs should be used to **empower people** , not to manipulate or harm them.

12.4 The Data Scientist of the Future

The role of the data scientist is going to change:

- Less time spent manually cleaning data.
- More time spent interpreting results.
- More focus on strategy and impact.
- Collaboration with intelligent agents
- Using LLMs as creative partners

12.5 A Vision for the Future

Imagine a scenario in 2030:

- You ask:

"Predict energy demand for the next 12 months and suggest public policies to reduce peak demand."
- The agent:
 1. It collects data on consumption, climate, and the economy.
 2. Creates temporal and contextual features.
 3. Testa Prophet, LSTM, and hybrid models



Predictive Analysis with Agents and LLMs in Action: Hands-On Python Practice with MYSQL on AWS | Azure | GCP

4. Choose the best one.
5. Generates predictions.
6. Detects blackout risks.
7. Suggests public policies
8. Creates executive and technical report.

All this in minutes. This is the future of predictive analytics.

12.6 Final Conclusion

We've reached the end of the journey. You've learned:

- Fundamentals of predictive analytics
- Data preparation
- Traditional models
- LLMs and intelligent agents
- Advanced feature engineering
- Time series
- Evaluation and monitoring
- Real cases
- The future of the area

Predictive analytics with agents and LLMs is not just a technique—it's a revolution. It transforms data into decisions, decisions into actions, and actions into real impact.

🌟 **Final message** : The future of predictive analytics isn't just about machines learning. It's about humans and agents working together to create a smarter, fairer, and more sustainable world.